Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



A Report on Lab-03

"Smart Text Editor: Observer Pattern and Memento Pattern"

Course Code: CSE 3206
Course Title: Software Engineering Sessional

Submitted By-

Md. Shuvo Mia – 2203052
Mahdi Hasen Ruan – 2203053
Tanzirul Islam – 2203054

Submitted To-

Emrana Kabir Hashi
Assistant Professor, Dept. of CSE, RUET

Date: 11/08/2026

1. Pattern Name

Observer Pattern and Memento Pattern (used together in one system: the Smart Text Editor)

2. Category

Behavioral Design Pattern (both patterns belong to this category)

3. Intent

This project uses two patterns in one editor.

The **Observer Pattern** lets the editor tell other parts of the program (like a word counter or a spell checker) whenever the text changes, without the editor knowing what those parts actually do.

The **Memento Pattern** lets the editor save a snapshot of its text and load it back later, giving the app an undo feature, without exposing the editor's private data.

4. Problem Statement

A text editor needs two things at the same time.

First, it needs to keep outside tools (word count, spell check) updated live as the user types, without hard-coding calls to each tool.

Second, it needs a safe way to undo changes, without letting other classes see or copy the editor's private text field directly.

Using one plain class for both jobs would mix unrelated logic together and make the code hard to read and extend.

Splitting the two jobs into two patterns keeps each responsibility clean and separate.

5. Motivation

Think of a simple text editor like Notepad, but smarter.

As the user types, a word counter and a spell checker should react right away – this is the Observer Pattern at work.

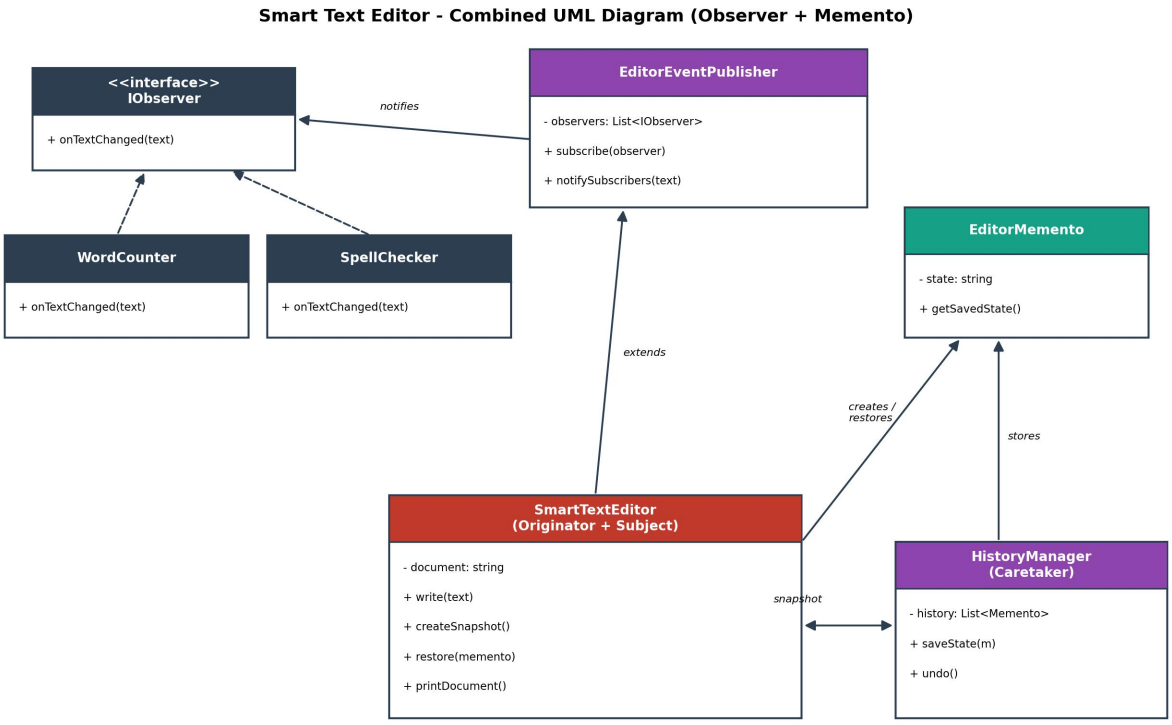
At the same time, every change should be saved as a snapshot, so the user can press undo and go back – this is the Memento Pattern at work.

In the code, one class, `SmartTextEditor`, plays two roles:

- It is the *Subject* that notifies observers.
- It is also the *Originator* that creates and restores snapshots.

Both patterns work side by side inside the same object.

6. Pattern Structure (UML Class Diagram)



7. Class Responsibilities

- **IObserver (Interface)** – Declares one method, `onTextChanged()`, that every observer must have.
- **WordCounter** – A concrete observer. Counts words each time the text changes and prints the count.
- **SpellChecker** – A concrete observer. Checks the text for simple typos and warns the user.
- **EditorEventPublisher (Subject)** – Keeps a list of observers. Allows observers to subscribe and notifies them when changes happen.
- **EditorMemento** – Stores a snapshot of the editor state.
- **HistoryManager (Caretaker)** – Maintains saved states and performs undo operations.
- **SmartTextEditor (Originator + Subject)** – Main class that manages document text, observer notification, and snapshot creation/restoration.

8. Code Implementation

```
1
2  #include <iostream>
3  #include <vector>
4  #include <memory>
5  #include <string>
6
7  using namespace std;
8
9
10 // ================= OBSERVER PATTERN =================
11
12
13 // Observer Interface
14 class IObserver {
15
16 public:
17
18     virtual void onTextChanged(const string& currentText) = 0;
19
20     virtual ~IObserver() = default;
21 };
22
23
24 // Concrete Observer 1
25 class WordCounter : public IObserver {
26
27 public:
28
29     void onTextChanged(const string& currentText) override {
30
31         int words = currentText.empty() ? 0 : 1;
32
33         for(char c : currentText)
34             if(c == ' ')
35                 words++;
36
37         cout << " -> [Plugin] Word Count: "
38              << words << endl;
39     }
40 };
41
42
43 // Concrete Observer 2
44 class SpellChecker : public IObserver {
45
46 public:
47
48     void onTextChanged(const string& currentText) override {
49
50         cout << " -> [Plugin] Spell Checker: Checking spelling..."
51              << endl;
```

```
52
53
54          if(currentText.find("helloo") != string::npos)
55
56              cout << "    [Warning] Typo detected: 'helloo'!"
57                   << endl;
58      }
59  };
60
61
62  // Subject
63  class EditorEventPublisher {
64
65
66  private:
67
68      vector<shared_ptr<IObserver>> observers;
69
70
71  public:
72
73
74      void subscribe(shared_ptr<IObserver> observer)
75      {
76          observers.push_back(observer);
77      }
78
79
80
81      void notifySubscribers(const string& text)
82      {
83          for(auto& obs : observers)
84              obs->onTextChanged(text);
85      }
86
87  };
88
89
90
91
92  // ================= MEMENTO PATTERN =================
93
94
95  // Memento
96  class EditorMemento {
97
98
99  private:
100
101     string state;
102
103
104 public:
105
```

```
106     EditorMemento(const string& text)
107         : state(text)
108     {}
109
110
111
112     string getSavedState() const
113     {
114         return state;
115     }
116
117 };
118
119
120
121
122 // Caretaker
123 class HistoryManager {
124
125
126 private:
127
128     vector<shared_ptr<EditorMemento>> history;
129
130
131
132 public:
133
134
135     void saveState(shared_ptr<EditorMemento> memento)
136     {
137         history.push_back(memento);
138     }
139
140
141
142     shared_ptr<EditorMemento> undo()
143     {
144
145         if(history.empty())
146             return nullptr;
147
148
149         history.pop_back();
150
151
152         if(history.empty())
153             return make_shared<EditorMemento>("");
154
155
156         return history.back();
157
158     }
159
```

```
};



// ================= MAIN EDITOR =================

// Originator + Subject
class SmartTextEditor : public EditorEventPublisher {

private:

    string document;


public:

    void write(const string& text)
    {

        document += text + " ";

        notifySubscribers(document);

    }


    shared_ptr<EditorMemento> createSnapshot()
    {

        return make_shared<EditorMemento>(document);

    }


    void restore(shared_ptr<EditorMemento> memento)
    {

        if(memento)

            document = memento->getSavedState();

        else

            document = "";


        notifySubscribers(document);
```

```
214
215     }
216
217
218
219     void printDocument()
220     {
221
222         cout << "Document: "
223              << document
224              << endl;
225
226     }
227
228 };
229
230
231
232
233 // ================= APPLICATION =================
234
235
236 int main()
237 {
238
239     SmartTextEditor editor;
240
241     HistoryManager history;
242
243
244
245     auto wordCounter =
246         make_shared<WordCounter>();
247
248
249     auto spellChecker =
250         make_shared<SpellChecker>();
251
252
253
254     editor.subscribe(wordCounter);
255
256     editor.subscribe(spellChecker);
257
258
259
260     history.saveState(
261         editor.createSnapshot()
262     );
263
264
265
266     cout << "Typing 'hello'..." << endl;
267
```

```
    editor.write("hello");


    history.saveState(
        editor.createSnapshot()
    );



    cout << "\nTyping 'worldd'..." << endl;

    editor.write("worldd");


    history.saveState(
        editor.createSnapshot()
    );



    cout << "\nCurrent Document State:"
         << endl;

    editor.printDocument();



    editor.restore(
        history.undo()
    );


    cout << "\nAfter Undo:"
         << endl;


    editor.printDocument();



    return 0;

}
```

9. Execution Flow (Object Interaction)

- The editor object is created.
- WordCounter and SpellChecker objects subscribe to the editor.
- When the user writes text, the editor updates its document and calls `notifySubscribers()`.
- All observers receive the updated text and perform their tasks. (Observer Pattern)
- After every modification, the editor creates a snapshot using `createSnapshot()`.
- The HistoryManager stores these snapshots. (Memento Pattern)
- When the user presses undo, HistoryManager removes the latest snapshot and returns the previous state.
- The editor restores the old state and notifies observers again.

10. Advantages

- The editor does not need to know the internal behavior of plugins.
- New observers can be added without modifying the editor class.
- Undo operation is implemented without exposing private document data.
- Responsibilities are separated, making the system easier to maintain.
- Both patterns improve code flexibility and scalability.

11. Limitations

- Too many observers may reduce performance because every change triggers all observers.
- Saving many snapshots can consume large memory.
- A very large editor class may become difficult to manage if more features are added.
- Restoring snapshots may trigger unnecessary observer updates.

12. Real-life Applications

- Text editors with spell checking and undo systems.
- Graphics software such as Photoshop and Figma.
- Collaborative document systems.
- Code editors like VS Code with live analysis and undo support.

13. Industry Examples

- Google Docs uses live updates for word count and comments (Observer-like behavior) and maintains document history (Memento-like behavior).
- Photoshop stores previous editing states and updates panels dynamically.
- Git maintains snapshots of project states while automation tools react to new commits.

14. Conclusion

The Smart Text Editor demonstrates how two behavioral design patterns can work together in a single application.

The Observer Pattern allows different components to react automatically whenever the document changes.

The Memento Pattern provides a safe mechanism for saving and restoring previous states without exposing internal data.

By combining these two patterns, the editor becomes easier to extend, maintain, and manage. New plugins can be added easily, and undo functionality can be implemented without creating unnecessary dependency between classes.